HMDA Mortgage Lending Fairness Pipeline

Detecting Racial and Geographic Disparities in U.S. Mortgage Lending (2023–2024)

Submitted by: Rohith Ambarish & Manoj Kumar Ravi Kumar

SEIS 745 - Data Lakes & Advanced Analytics

GitHub repo: HDMA-DataLake-Pipeline

Dataset Description and Motivation

For this project, we selected the Home Mortgage Disclosure Act (HMDA) dataset, a federally mandated collection of mortgage loan application records published annually by the Federal Financial Institutions Examination Council (FFIEC). The dataset covers every mortgage application filed with reporting lenders across the United States, including detailed fields on the applicant's race, ethnicity, income, loan amount, interest rate, rate spread, and the lender's final decision on the application. We focused on two years of data, i.e., 2023 and 2024, across ten of the most populous U.S. states: California, Texas, Florida, New York, Pennsylvania, Illinois, Ohio, Georgia, North Carolina, and Minnesota (Explicitly considered this because we're right now).

What drew us to this dataset was the intersection of big data scale and genuine social impact. HMDA data is one of the most important tools available to regulators, researchers, and community advocates for identifying discriminatory lending patterns. With approximately 6 million records per year and 99 fields per record, the combined two-year dataset exceeded 11.7 million rows and 4.4 gigabytes, a scale that renders standard analytical tools like Excel or single-machine Python scripts completely impractical. This made it a perfect/natural fit for a distributed computing course project. Beyond the technical challenge, the subject matter is compelling: mortgage lending decisions determine whether families can build wealth through homeownership, and disparities in those decisions have documented long-term economic consequences for minority communities.

Data Collection: Approach and Challenges

The FFIEC provides public access to HMDA data through its Data Browser API, which allows programmatic download of state-level CSV files by year. Rather than manually downloading files through the browser interface, we built a Python collection script using the requests library. The script incorporated retry logic with exponential backoff to handle transient network failures, resume support to skip files already downloaded, rate limiting to avoid overloading the government API, and structured logging with timestamps to monitor progress across all twenty download jobs.

The most significant challenge in data collection was storage. As we know AWS Cloud9 EC2 instance provides only 10 gigabytes of local disk by default, and the 2023 data alone filled it completely before we could begin downloading 2024 data. We evaluated two options: expanding the EC2 disk to 200 gigabytes, or redirecting the data to Amazon S3. We chose S3 as we can't even limit this to 200 gigabytes, which is still less if we expand the dataset. We rewrote the download script to stream API responses directly into S3 using boto3's multipart upload API, which allowed us to transfer files chunk by chunk without materializing the full file on local disk. The 2024 data was uploaded to S3 successfully using this approach, while the 2023 files that had already been downloaded locally were transferred to S3 using the AWS CLI.

Data Preparation: Steps and Challenges

Data preparation was executed across two Databricks notebooks using Apache Spark on serverless compute. The first step was ingestion: reading all twenty CSV files into a single Spark DataFrame using an explicit 99-field schema we defined using Spark's StructType API. Defining the schema explicitly rather than allowing Spark to infer it was critical for two reasons. First, schema inference requires a full scan of the dataset just to determine column types, which is wasteful at this scale. Second, HMDA data contains placeholder values such as "NA" and "Exempt" in numeric columns, which would cause Spark to incorrectly infer those columns as string types.

A significant challenge was when we discovered that our initial schema had an off-by-one column alignment error. The HMDA CSV files contained 99 columns, but our schema defined only 98, having missed the tract_minority_population_percent field at column index 93. This caused all downstream census tract columns to be misaligned, producing values which made no sense, the tract_to_msa_income_percentage column, which should contain percentage values between zero and two hundred, was instead showing dollar amounts in the ninety thousand range. After diagnosing the issue by reading the raw CSV headers, we corrected the schema and re-ran the ingestion pipeline from scratch.

The cleaning phase involved type casting numeric columns from strings using Spark's try_cast expression rather than the standard cast function, which would crash on "NA" values. We filtered the dataset to the three action codes relevant to our analysis, originated, approved but not accepted, and denied, removing withdrawn and incomplete applications. Outlier removal was applied to interest rates outside the range of one to twenty percent, loan-to-value ratios above two hundred percent, and property values that matched the integer overflow sentinel value of 2,147,483,647. After filtering, the dataset was reduced from 11.7 million rows to approximately 8 million clean records. The enrichment phase added derived columns including a binary denial flag, income bracket categories, a simplified race group variable, LMI tract classification, and a minority tract flag based on the census tract's minority population percentage.

Data Analysis: Approach, Insights, and Challenges

All analytical queries were executed using Spark SQL. We structured the analysis around five research questions, using common table expressions, GROUP BY aggregations, window functions, and cross-joins to compute disparity gaps. The gold Delta table was never re-read from raw CSV during analysis, all queries ran against the pre-computed checkpoint, which made iterative development fast and reliable.

The most striking finding from the denial rate analysis was the persistence of the racial gap across every income bracket. A high-income Black applicant earning more than $250,000 annually faced a denial rate of 29.02 percent, compared to 15.23 percent for a white applicant in the same income bracket. This means a high-income Black applicant was denied at nearly twice the rate of a similarly situated white applicant. The gap did not diminish with income, in fact, for Native American applicants, the ratio at the highest income bracket reached 1.98 times the white denial rate. This finding is significant because income is the most commonly cited explanation for lending disparities. The data shows it is not sufficient to explain the observed differences.

The lender fairness analysis produced equally important findings. Among the top 50 lenders by origination volume, Citibank showed the largest denial gap: minority applicants were denied at 42.42 percent compared to 17.30 percent for white applicants, a difference of 25.12 percentage points. Kiavi Funding showed a different form of market exclusion, zero percent of its 26,615 applications came from minority borrowers, suggesting systematic geographic or product targeting that excludes minority

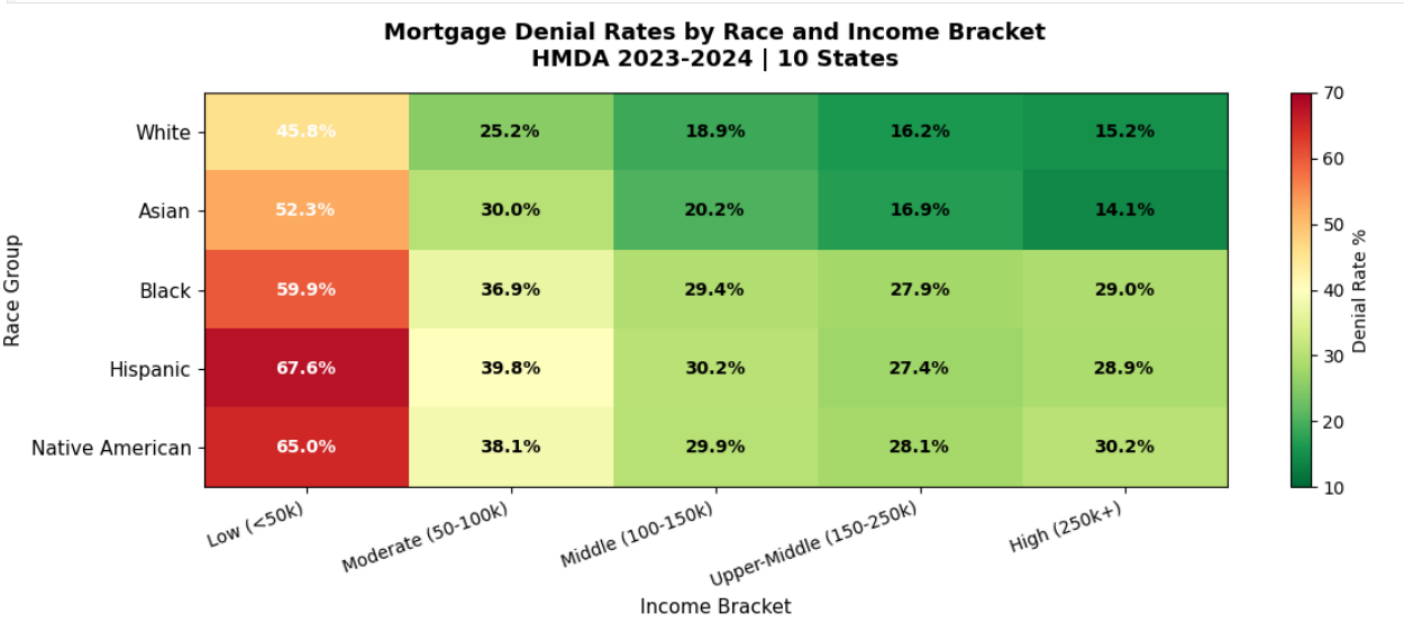
communities entirely. These are not small regional lenders. these are major national institutions whose lending patterns affect hundreds of thousands of families annually.

The primary challenge in analysis was designing queries that controlled for confounding variables. Comparing denial rates across racial groups without controlling for income would conflate the effects of race and income. We addressed this by mapping all comparisons within income brackets and using CTEs to compute white applicant baseline rates for explicit gap calculations.

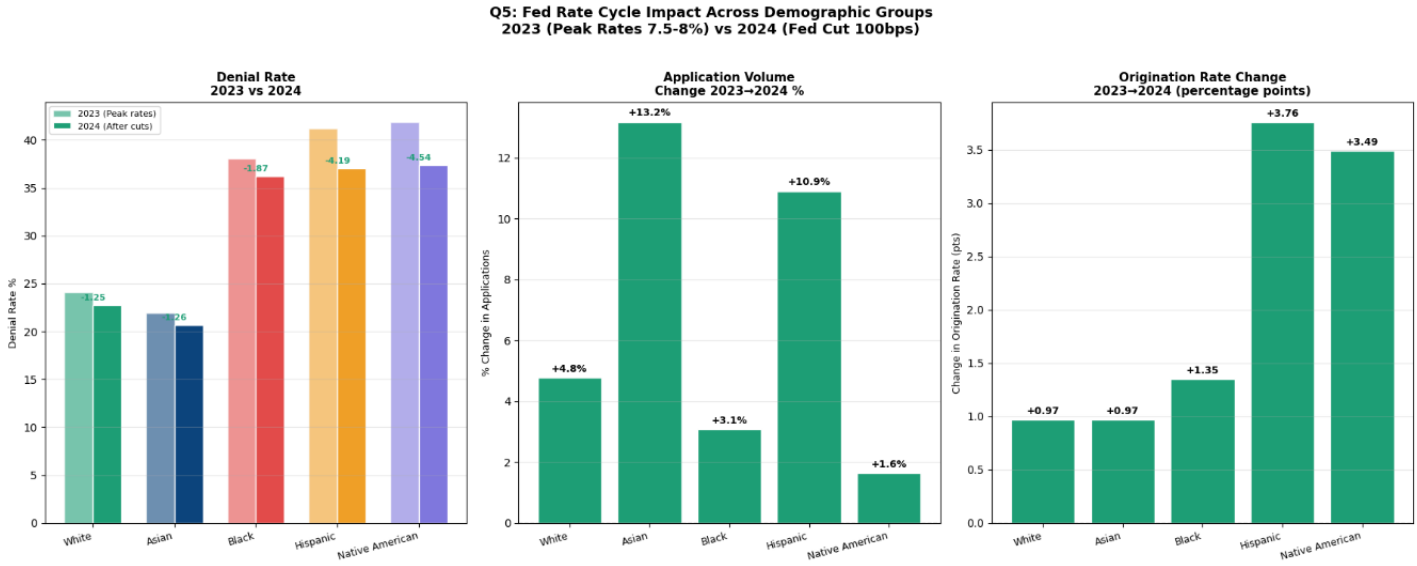
Visualization: Approach, Descriptions, and Challenges

Visualizations were created using matplotlib in Databricks notebooks, converting Spark DataFrames to pandas for plotting. We produced seven charts covering all five research questions. Charts were saved to a Databricks Volume as PNG files and subsequently exported to GitHub repository.

The denial rate heatmap was one of the most effective visualizations in the project. It displayed denial rates as a two-dimensional color matrix with racial groups on the vertical axis and income brackets on the horizontal axis, using a red-to-green color scale. The visual immediately communicates the central finding of the project: the matrix is strikingly red for Black, Hispanic, and Native American rows at every income level, while the White and Asian rows remain comparatively green. A viewer does not need to read any numbers to understand that something is systematically different about how different racial groups experience the mortgage application process.



The Fed rate cycle chart compared 2023 and 2024 outcomes across racial groups, showing that while minority borrowers improved more than white borrowers in denial rate reduction, Native American applicants improved by 4.54 percentage points compared to 1.25 points for white applicants, the absolute denial rate gap remained unchanged at approximately 13 to 14 percentage points. The visualization made the nuance of this finding accessible, improvement occurred, but the structural gap did not close.



The main visualization challenge was working within the constraints of Databricks Serverless compute, which does not support the standard Spark caching API. This required us to be deliberate about minimizing re-computation by always reading from Delta checkpoints rather than re-running Spark transformations during the visualization phase. A secondary challenge was the initial schema misalignment described in the preparation section, which caused our first attempt at the credit desert scatter plot to show nonsensical census tract income values until the underlying data was corrected.

Overall Reflection

This project exceeded our initial expectations in both technical complexity and analytical depth. Several real-world engineering challenges, disk exhaustion, schema misalignment, Databricks Serverless API restrictions, AWS credential rotation, required us to adapt our approach repeatedly rather than following our initial predetermined plan. These challenges were valuable precisely because they reflected the kinds of problems that arise in production data engineering work, where infrastructure limitations, data quality issues, and platform constraints are constant realities.

The decision to use S3 as the data lake rather than expanding local EC2 storage was the most important architectural choice we made. It forced us to think about the pipeline in terms of cloud-native design principles, separating compute from storage, using streaming uploads to avoid memory constraints, and treating the Databricks Volume as an intermediary staging layer rather than a permanent home for raw data. In contrast, this approach was more complex to implement but resulted in a more realistic and transferable architecture.

The Delta Lake medallion architecture, Bronze for raw ingested data, Silver for cleaned data, and Gold for enriched analytical data proved extremely valuable. When we discovered the schema misalignment error midway through the project, we were able to re-run only the ingestion and cleaning notebooks without touching the analysis or visualization code.

If we were to approach this project again, there are several things we'll do differently. First, we'll validate the column count and alignment of the raw CSV headers before building the schema, which would have caught the off-by-one error before any downstream work was done. Second, we would set up a more automated pipeline using Apache Airflow for orchestration rather than running notebooks manually in sequence, which would make the pipeline easier to re-execute on a schedule or in response to new HMDA data releases. Third, we would add a data quality validation layer between the Silver and Gold tables using a framework such as Great Expectations, which would provide automated checks for null

rates, value ranges, and referential integrity rather than relying on manual spot checks. Finally, the lender LEI-to-name lookup could be incorporated earlier in the pipeline rather than being executed as a post-hoc API call during visualization, which would make lender names available throughout the analysis phase.

Overall, this project demonstrated that distributed data engineering tools are not just appropriate for academic exercises, they are necessary for working with data at realistic scales, and the insights they enable have genuine policy relevance. The finding that a high-income Black applicant faces nearly twice the denial rate of a high-income white applicant is not a statistical artifact. It is a signal embedded in federally reported data that regulators, journalists, and community organizations can and should use to drive accountability in mortgage lending.

References:

- FFIEC, *HMDA Loan Application Register, 2023 Public Data*, https://ffiec.cfpb.gov/data-browser/data/2023.
- FFIEC, *HMDA Loan Application Register, 2024 Public Data*, https://ffiec.cfpb.gov/data-browser/data/2024.
- FFIEC, *FFIEC Census and Demographic Data*, https://www.ffiec.gov/censusapp.htm.